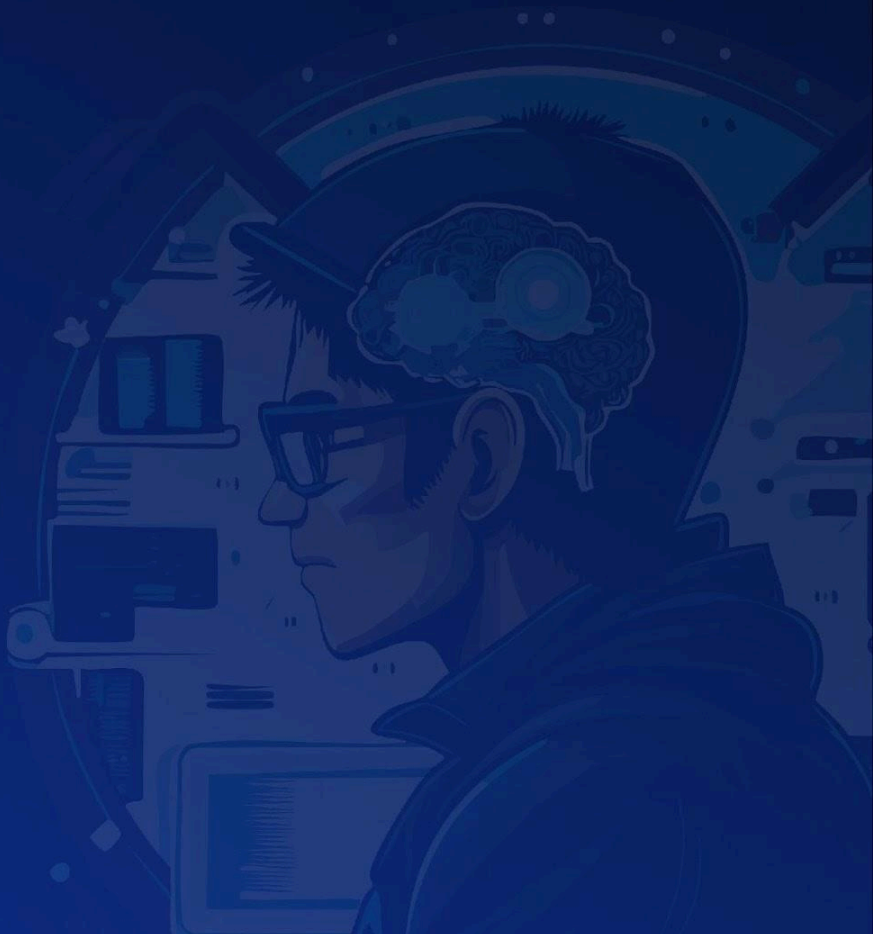
Cypher Launchpad

Security Review

Disclaimer

Security Review

Cypher Launchpad



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

Cypher Launchpad

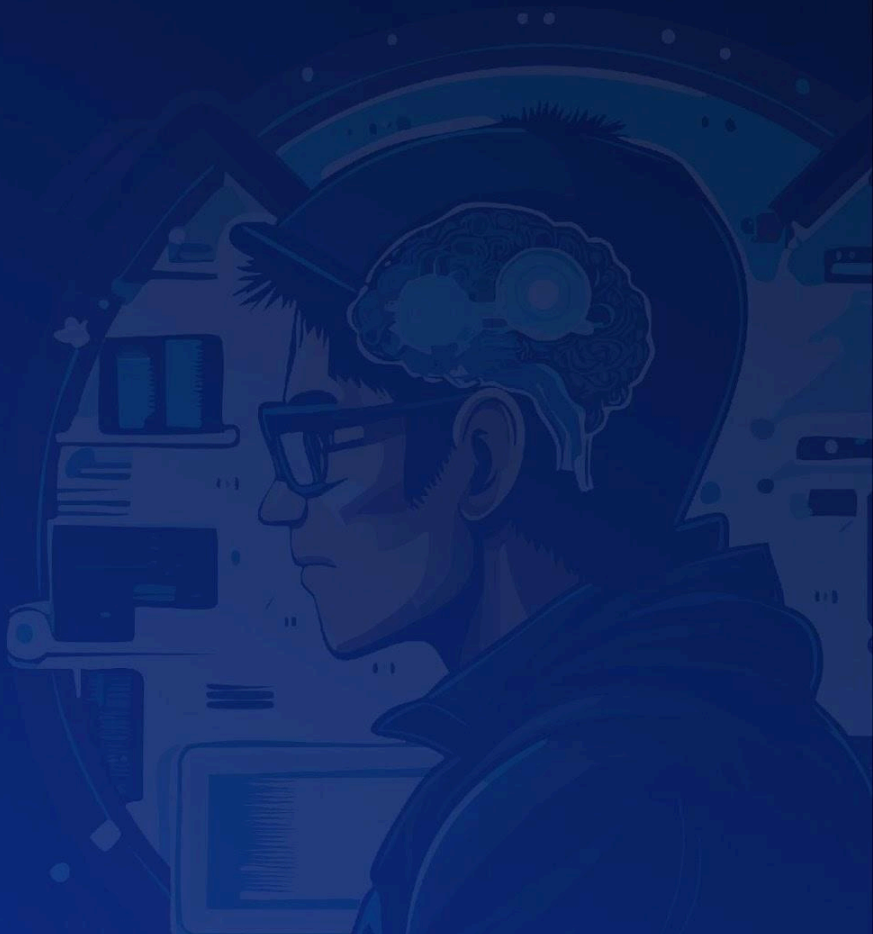
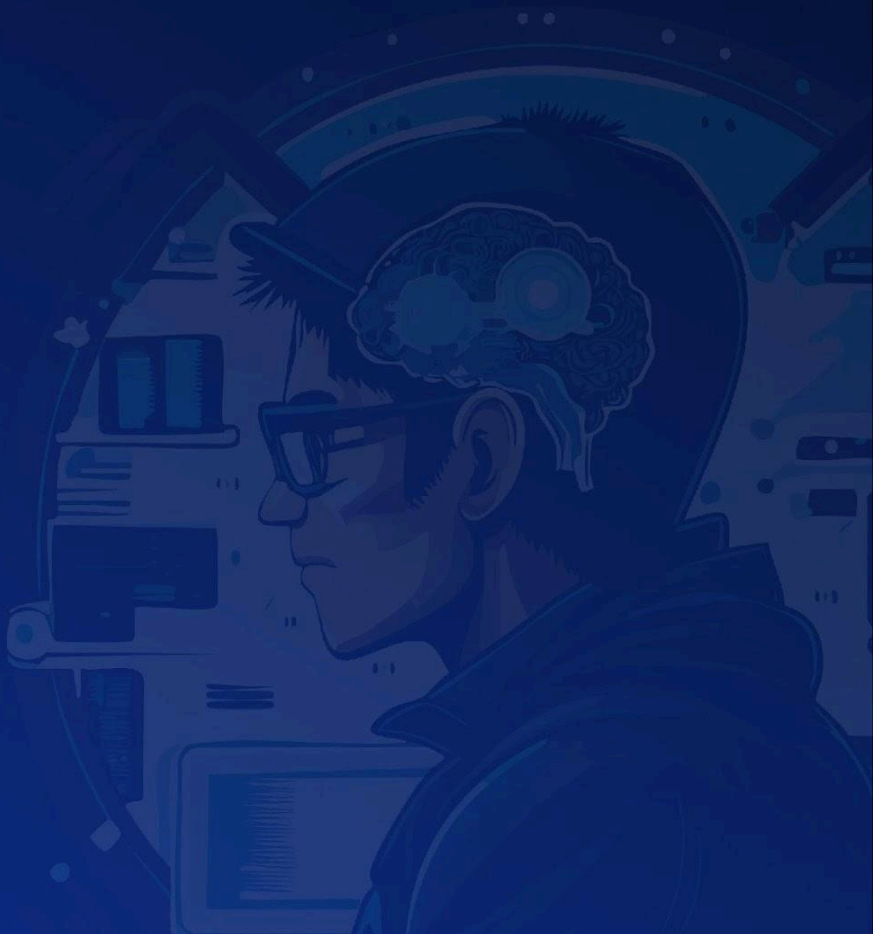


Table of Contents

Summary

Security Review

Cypher Launchpad



Summary

Three Sigma audited Camelot in a 1.2 person week engagement. The audit was conducted from 13/01/2026 to 15/01/2026.

Protocol Description

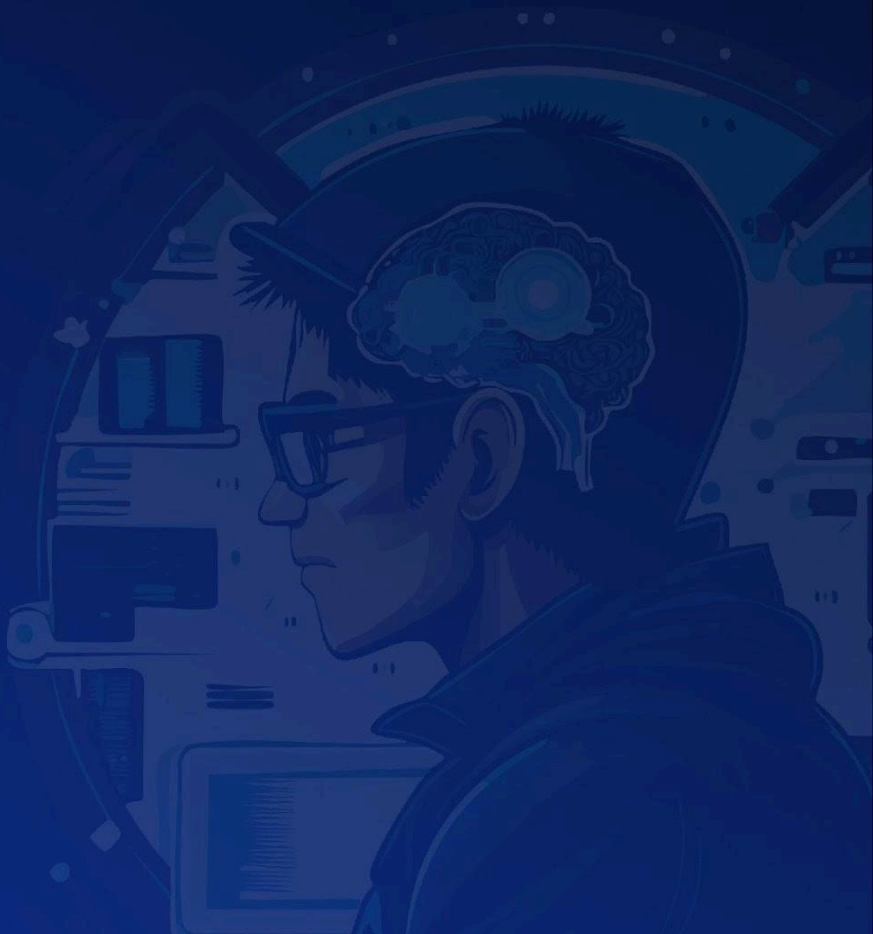
Gu is an Arbitrum-based token factory built on top of Camelot DEX. Whenever a new token is launched, a new Camelot Liquidity Pool is created, setting its `sqrtPriceX96` and it enters a _Bonding Curve_ phase where its params are inherited from the `GuCoinFactory.sol` config. During the _Bonding Curve_ phase, users are able to trade within a virtual bonding curve until the token `MAX_SUPPLY` is reached (which is equivalent to reaching a target market cap or `TARGET_ETH`).

Once the `MAX_SUPPLY` is reached, all the ETH available plus an initial gu coin supply are deployed as liquidity to the created LP and the positions are transferred to a `Harvester` contract which locks them but enables a privileged account or contract to harvest the fees. GuCoin supply is deployed from the _seed tick_ to MAX or MIN tick and ETH is deployed from _seed tick_ to the _initial expected tick_ which matches the first price of the Bonding Curve.

Scope

Security Review

Cypher Launchpad



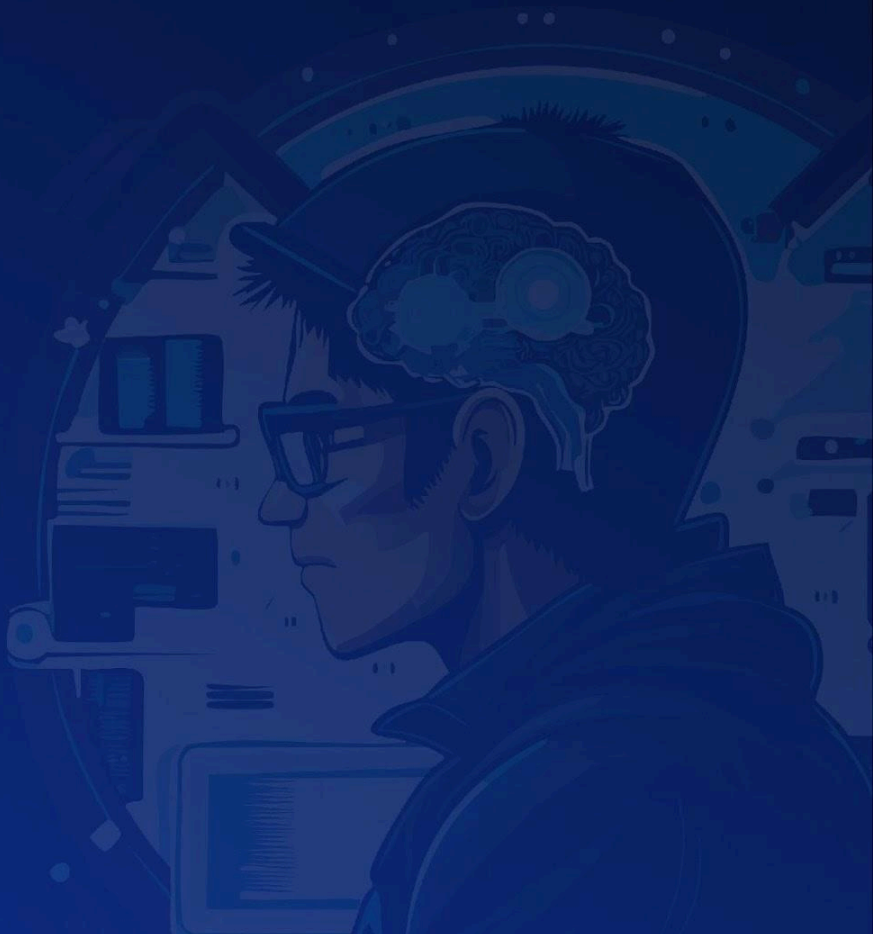
Scope

Filepath	nSLOC
src/BondingCurve.sol	171
src/BCTokenFactory.sol	168
src/ReferralManager.sol	142
src/StakingVault.sol	112
Total	**593**

Methodology

Security Review

Cypher Launchpad



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

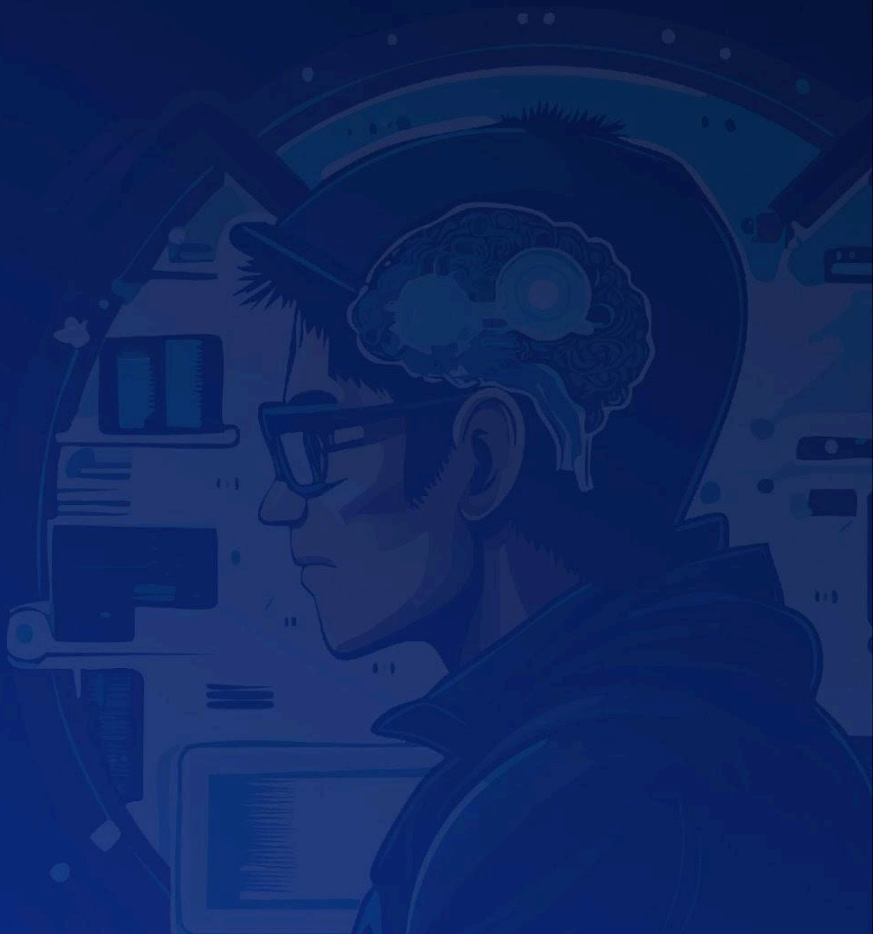
In this audit, we classify findings based on Immunefi's Vulnerability Severity Classification System (v2.3) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

Cypher Launchpad



Project Dashboard

Application Summary

Name	Camelot
Repository	https://github.com/CypherIndustries/factory-contracts
Commit	9cf8c2c
Language	Solidity
Platform	Arbitrum

Engagement Summary

Timeline	13/01/2026 to 15/01/2026
№ of Auditors	2
Review Time	1.2 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	1	1	0
Low	4	3	1
None	2	1	1

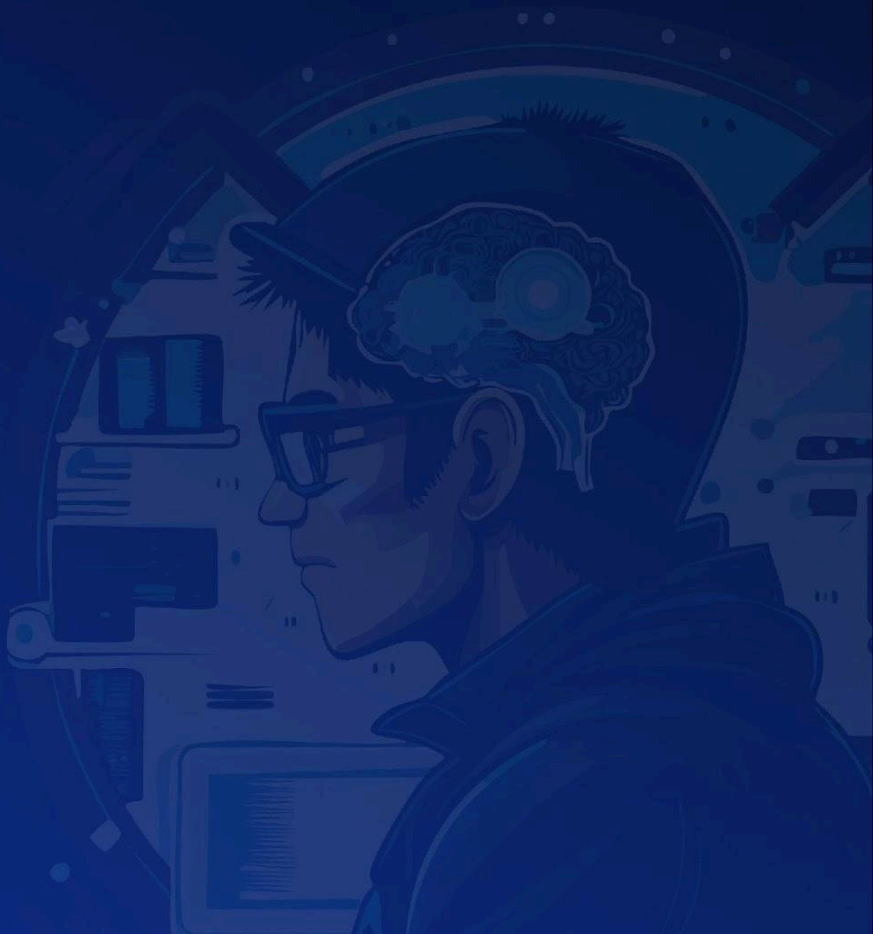
Category Breakdown

Suggestion	2
Documentation	0
Bug	5
Optimization	0
Good Code Practices	0

Risk Section

Security Review

Cypher Launchpad



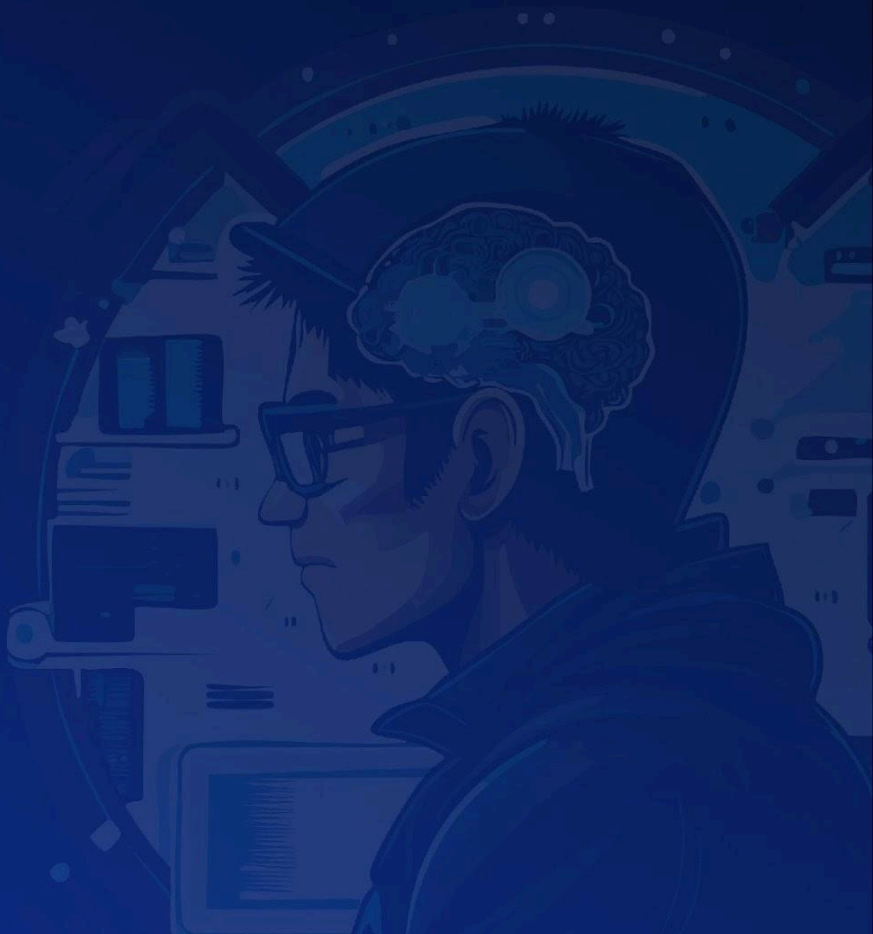
Risk Section

No risks identified.

Findings

Security Review

Cypher Launchpad



Findings

3S-Launchpad-M01

Shared referral expiration timestamp causes inconsistent referral periods across tokens

Id	3S-Launchpad-M01
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #0ab07d9.

Description

The **ReferralManager** contract manages referral rewards for users who bring new participants to the protocol. When a user buys or sells tokens through the **BondingCurve**, the **_handleReferralFees** function distributes a portion of the protocol fee to referrers. The **validUntil** variable determines the deadline after which referral rewards can no longer be accumulated.

The protocol architecture establishes a shared dependency chain: multiple **BCToken** instances deployed via **BCTokenFactory::deploy** all reference the same **bondingCurve** address stored in the factory. This **BondingCurve** contract holds a single **referralManager** address, and the **ReferralManager** contract maintains a single **validUntil** timestamp that governs referral validity for the entire system.

This design means that all tokens, regardless of their individual deployment timestamps, are subject to the same referral expiration. A token deployed near the end of the referral period will have minimal time to benefit from referral incentives, while tokens deployed after **validUntil** will have no referral period at all.

Steps to reproduce

1. Deploy the protocol with **validUntil** set to 30 days from now
2. Deploy **TokenA** on day 1 via **BCTokenFactory::deploy**
3. **TokenA** has 29 days of referral incentives remaining
4. Deploy **TokenB** on day 25 via **BCTokenFactory::deploy**

5. **TokenB** only has 5 days of referral incentives, despite being newly created
6. Deploy **TokenC** on day 31 via **BCTokenFactory::deploy**
7. **TokenC** has no referral period since **validUntil** has already passed
8. Calls to **ReferralManager::receiveReferral** for **TokenC** will revert with **ReferralExpired**

Recommendation

Deploy a dedicated **ReferralManager** instance for each **BCToken**. The **BondingCurve** should maintain a mapping of tokens to their respective referral managers and deploy new instances when tokens are created.

IBondingCurve.sol - Add new interface functions:

```
interface IBondingCurve {
    // ... existing declarations ...
    /// @notice Returns the referral manager for a specific token
    function tokenReferralManagers(address token) external view returns (address);
    /// @notice Deploys a new referral manager for a token
    function deployReferralManager(address token) external returns (address);
}
```

BondingCurve.sol - Replace single referral manager with per-token mapping:

```
- /// @inheritdoc IBondingCurve
- address public referralManager;
+ /// @notice Mapping of token addresses to their referral managers
+ mapping(address => address) public tokenReferralManagers;
+ /// @notice Configuration for deploying new referral managers
+ uint256 public referralDuration;
+ uint256 public directRefFeeBps;
+ uint256 public indirectRefFeeBps;
+ address[] public payoutTokens;
+ uint256[] public payoutThresholds;
```

```
function buy(address token, address affiliate, uint256 expectedOut)
    external
    payable
    nonReentrant
    returns (uint256)
```

```
 {
     _validateToken(token);
     if (factory.PROTOCOL_PAUSED()) revert ProtocolPaused();
     if (msg.value == 0) revert InvalidAmount();
+     address referralManager = tokenReferralManagers[token];
     if (affiliate != address(0) && referralManager != address(0)) {
         IReferralManager(referralManager).setReferral(affiliate, msg.sender);
     }
     // ... existing buy logic ...
     // Handle referral fees if referral manager is set.
-     uint256 referralFee = _handleReferralFees(msg.sender, fee);
+     uint256 referralFee = _handleReferralFees(token, msg.sender, fee);
     // ... rest of function ...
 }


 function sell(address token, address affiliate, uint256 amount, uint256 expectedOut)
     external
     nonReentrant
     returns (uint256)
 {
     _validateToken(token);
     if (factory.PROTOCOL_PAUSED()) revert ProtocolPaused();
     if (amount == 0) revert InvalidAmount();
+     address referralManager = tokenReferralManagers[token];
     if (affiliate != address(0) && referralManager != address(0)) {
         IReferralManager(referralManager).setReferral(affiliate, msg.sender);
     }
     // ... existing sell logic ...
     // Handle referral fees if referral manager is set.
-     uint256 referralFee = _handleReferralFees(msg.sender, fee);
+     uint256 referralFee = _handleReferralFees(token, msg.sender, fee);
     // ... rest of function ...
 }


- function _handleReferralFees(address user, uint256 protocolFee) internal returns (uint256
referralFee) {
+ function _handleReferralFees(address token, address user, uint256 protocolFee) internal
returns (uint256 referralFee) {
+     address referralManager = tokenReferralManagers[token];
     if (referralManager == address(0) || protocolFee == 0) return 0;
     referralFee = IReferralManager(referralManager).quoteReferralFee(user, protocolFee);
```

```
    if (referralFee == 0) return 0;
    // Convert ETH to WETH.
    IWETH(weth).deposit{value: referralFee}();
    // Approve and send to referral manager.
    IWETH(weth).approve(referralManager, referralFee);
    IReferralManager(referralManager).receiveReferral(user, weth, referralFee);
}
```

```
+ /// @notice Deploys a new referral manager for a token
+ /// @param token The token address to deploy the referral manager for
+ /// @return manager The address of the deployed referral manager
+ function deployReferralManager(address token) external returns (address manager) {
+     _validateToken(token);
+     if (tokenReferralManagers[token] != address(0)) revert
ReferralManagerAlreadyDeployed();
+
+     manager = address(new ReferralManager(
+         directRefFeeBps,
+         indirectRefFeeBps,
+         block.timestamp + referralDuration,
+         payoutTokens,
+         payoutThresholds
+     ));
+
+     IReferralManager(manager).setWhitelistedBondingCurve(address(this), true);
+     tokenReferralManagers[token] = manager;
+
+     emit ReferralManagerDeployed(token, manager);
+ }
+ /// @notice Updates the referral configuration for future deployments
+ function setReferralConfig(
+     uint256 _referralDuration,
+     uint256 _directRefFeeBps,
+     uint256 _indirectRefFeeBps,
+     address[] calldata _payoutTokens,
+     uint256[] calldata _payoutThresholds
+ ) external {
+     _validateFactoryOwner();
+     referralDuration = _referralDuration;
+     directRefFeeBps = _directRefFeeBps;
+     indirectRefFeeBps = _indirectRefFeeBps;
+     payoutTokens = _payoutTokens;
```

```
+     payoutThresholds = _payoutThresholds;
+ }
- function setReferralManager(address _referralManager) external {
-     _validateFactoryOwner();
-
-     referralManager = _referralManager;
-     emit ReferralManagerUpdated(_referralManager);
- }
```

Moreover, add delegating setters in **BondingCurve** that forward calls to each token's referral manager: **setTokenDirectRefFeeBps**, **setTokenIndirectRefFeeBps**, **setTokenValidUntil**, **setTokenPayoutThreshold**, and **setTokenWhitelistedBondingCurve**

BCTokenFactory.sol - Deploy referral manager after token creation:

```
  function deploy(
      string memory name,
      string memory symbol,
      string memory description,
      string memory image,
      uint8 communityFeeRatio,
      bytes32 salt
  ) external payable returns (address) {
      // ... existing validation ...
      address token = _deployToken(name, symbol, description, image, communityFeeRatio,
salt);
      bcTokensParams[params] = token;
      bcTokens[token] = true;
+     // Deploy a dedicated referral manager for this token
+     bondingCurve.deployReferralManager(token);
      // ... rest of function ...
  }
```

3S-Launchpad-L01

Hash collision in **bcTokensParams** due to **abi.encodePacked**

Id	3S-Launchpad-L01
Classification	Low
Impact	Low
Likelihood	Medium
Category	Bug
Status	Addressed in #131a4c4.

Description

In **BCTokenFactory.sol**, **abi.encodePacked** is used to generate a unique hash from the parameters **name**, **symbol**, **description**, and **image**:

```
bytes32 params = keccak256(abi.encodePacked(name, symbol, description, image));
```

This hash is stored in the **bcTokensParams** mapping to prevent duplicate deployments with identical parameters.

The problem is that **abi.encodePacked** with multiple dynamic types (strings) is collision-prone. For example, **abi.encodePacked("ab", "c")** produces an identical result to **abi.encodePacked("a", "bc")** since the concatenation is the same.

This means different, valid token parameters can produce the same hash, causing legitimate deployments to fail with **InvalidParams()** revert even though the actual data is unique.

Recommendation

Replace **abi.encodePacked** with **abi.encode**, which adds padding and length prefixes for each dynamic type, eliminating the possibility of collisions:

```
- bytes32 params = keccak256(abi.encodePacked(name, symbol, description, image));
+ bytes32 params = keccak256(abi.encode(name, symbol, description, image));
```

3S-Launchpad-L02

CREATE3 deployment can be frontrun due to salt not including **msg.sender**

Id	3S-Launchpad-L02
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #bfb1eed.

Description

In **BCTokenFactory.sol**, the **salt** parameter is passed directly to the CREATE3 factory without including **msg.sender**:

```
ICREATE3Factory(CREATE_DEPLOYER).deploy(salt, bytecode);
```

With CREATE3, the deployed contract address depends only on the deployer address and the salt. Since all deployments go through the same **BCTokenFactory** contract, the final address is determined solely by the user-provided salt.

This allows an attacker to monitor the mempool for pending deploy transactions, extract the salt, and submit their own transaction with the same salt and higher gas price. The attacker's transaction gets mined first, and the original user's transaction fails with **SaltAlreadyUsed()**.

This can block users from deploying tokens with a specific salt of their choice.

Recommendation

Include **msg.sender** in the salt to create a separate namespace for each user:

```
- ICREATE3Factory(CREATE_DEPLOYER).deploy(salt, bytecode);
+ ICREATE3Factory(CREATE_DEPLOYER).deploy(keccak256(abi.encodePacked(msg.sender, salt)), bytecode);
```

Another possible mitigation is to increase the **_creationFee**.

3S-Launchpad-L03

Referral affiliate cannot be updated per token due to shared referral manager

Id	3S-Launchpad-L03
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Acknowledged

Description

The **BondingCurve::buy** and **BondingCurve::sell** functions allow users to specify an **affiliate** address that will receive referral rewards from their trades. When provided, the function calls **ReferralManager::setReferral** to establish the referral relationship. However, **ReferralManager::setReferral** only sets the referral if the user does not already have one, as indicated by the condition **referredBy[user] != address(0)**.

Since all **BCToken** instances share the same **ReferralManager**, a user's referral is set globally across all tokens upon their first trade with an affiliate. Any subsequent attempt to specify a different affiliate for a different token is silently ignored.

This design prevents users from choosing different affiliates for different tokens and locks them into a permanent referral relationship based solely on their first trade.

Steps to reproduce

1. Alice calls **BondingCurve::buy** for **BCToken1** with **affiliate** set to Bob

2. **ReferralManager::setReferral** is called, setting **referredBy[Alice] = Bob**

3. Later, Alice calls **BondingCurve::buy** for **BCToken2** with **affiliate** set to Charlie

4. **ReferralManager::setReferral** is called, but since **referredBy[Alice]** is already Bob, the function returns early without updating

5. Charlie receives no referral rewards from Alice's **BCToken2** trades, and Bob continues to receive rewards despite not being the intended affiliate

---

Recommendation

This issue is resolved by the fix proposed in the report #13. By deploying a dedicated **ReferralManager** for each **BCToken**, users can establish independent referral relationships per token. Each token's referral manager maintains its own **referredBy** mapping, allowing users to specify different affiliates for different tokens.

3S-Launchpad-L04

Inconsistent fee calculation in **BondingCurve::buy** when token supply cap is reached

Id	3S-Launchpad-L04
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Addressed in #0fe1c53.

Description

The **BondingCurve::buy** function allows users to purchase tokens from the bonding curve by sending ETH. The function calculates a transaction fee based on the **txFee** percentage and uses the remaining ETH as the cost for purchasing tokens.

The **_calculateFee** function computes the fee as **(amount * txFee) / 10_000**. When applied to **msg.value**, this treats the fee as a percentage of the total input rather than the effective cost. However, when the supply cap is reached, the function recalculates the fee using **_calculateFee(cost)**, which correctly treats the fee as a percentage of the cost.

This creates an inconsistency between the two code paths:

- **Normal case**: **fee = (msg.value * txFee) / 10_000** and **cost = msg.value - fee**. The fee is a percentage of the total input.

- **Supply cap reached**: **fee = (cost * txFee) / 10_000**. The fee is a percentage of the cost.

This results in two impacts:

1. Users pay a lower effective fee rate when the supply cap is reached compared to normal purchases.

2. Because the normal case calculates the fee as a percentage of the total input rather than the cost, the effective fee rate on the cost exceeds the intended **MAX_FEE** limit. For example, with **txFee** set to **MAX_FEE** (500 basis points or 5%), a user sending 100 ETH would pay a fee of 5 ETH on a cost of 95 ETH, resulting in an effective fee rate of approximately 5.26% on the cost.

Steps to reproduce

1. Assume **txFee** is set to 10% (1000 basis points).

2. User A sends 100 ETH when **MAX_SUPPLY** is not reached:

   - **fee = (100 * 1000) / 10_000 = 10 ETH**

   - **cost = 100 - 10 = 90 ETH**

   - Fee as percentage of cost: 10 / 90 ≈ 11.11%

3. User B sends 100 ETH when **MAX_SUPPLY** is reached, and the recalculated **cost** is 90 ETH:

   - **fee = (90 * 1000) / 10_000 = 9 ETH**

   - Fee as percentage of cost: 9 / 90 = 10%

User B pays a lower fee than User A for the same effective cost.

Recommendation

Modify **_calculateFee** to derive the fee such that it represents a percentage of the cost (not the total input). This aligns with the **MAX_FEE** constant, which defines the maximum fee as a percentage of the transaction cost. When the supply cap is reached, apply the fee directly to the recalculated cost using the standard formula.

```
    function buy(address token, address affiliate, uint256 expectedOut)
        external
        payable
        nonReentrant
        returns (uint256)
    {
        // ... existing code ...
        if (amountOut > supplyLeft) {
            amountOut = supplyLeft;
            cost = CurveMath(formula).purchaseCost(
                supplyLeft + coin.INITIAL_SUPPLY(),
                coin.reserveBalance(),
                reserveWeight,
                amountOut
            );
-           fee = _calculateFee(cost);
+            fee = txFee > 0 ? (cost * txFee) / 10_000 : 0;
            (bool ok,) = payable(msg.sender).call{value: msg.value - cost - fee}("");
            if (!ok) revert FailedToSendETH();
        }

        // ... existing code ...
```

```
    }
    function _calculateFee(uint256 amount) internal view returns (uint256) {
-       return txFee > 0 ? (amount * txFee) / 10_000 : 0;
+        return txFee > 0 ? (amount * txFee) / (10_000 + txFee) : 0;
    }
```

3S-Launchpad-N01

Unused **reserveWeight** parameter in bonding curve calculations

Id	3S-Launchpad-N01
Classification	None
Category	Suggestion
Status	Acknowledged

Description

In **BondingCurve.sol**, the **reserveWeight** parameter is passed to all calculation functions (**purchaseCost**, **purchaseTargetAmount**, **saleTargetAmount**):

```
return CurveMath(formula).purchaseTargetAmount(
    coin.MAX_SUPPLY() - coin.totalSupply() + coin.INITIAL_SUPPLY(),
coin.reserveBalance(), reserveWeight, amount
);
```

However, the parameter has no effect on the bonding curve's pricing behavior. It is only used for validation but never included in the actual calculations. This is misleading as it suggests the parameter influences the curve shape when it does not.

Recommendation

Remove the unused parameter to avoid confusion

3S-Launchpad-N02

Redundant token validation in buy and sell functions

Id	3S-Launchpad-N02
Classification	None
Category	Suggestion
Status	Addressed in #4b9273e.

Description

The **BondingCurve** contract facilitates buying and selling tokens along a bonding curve. The **buy** function allows users to purchase tokens by sending ETH, while the **sell** function allows users to sell tokens back to the curve in exchange for ETH. Both functions include a **_validateToken** check to ensure the provided token address is a valid **BCToken** created by the factory.

However, the initial **_validateToken(token)** call at the beginning of both **BondingCurve::buy** and **BondingCurve::sell** is redundant. This is because both functions subsequently call **getAmountOutBuy** and **getAmountOutSell** respectively, which internally perform the same **_validateToken(token)** check.

This results in duplicate storage reads to **factory.bcTokens(token)**, wasting gas on every buy and sell operation.

Recommendation

Remove the redundant **_validateToken(token)** calls from the beginning of **BondingCurve::buy** and **BondingCurve::sell** functions since the validation is already performed within **getAmountOutBuy** and **getAmountOutSell**.

```
 function buy(address token, address affiliate, uint256 expectedOut)
     external
     payable
     nonReentrant
     returns (uint256)
 {
-    _validateToken(token);
-
     if (factory.PROTOCOL_PAUSED()) revert ProtocolPaused();
```

```
        if (msg.value == 0) revert InvalidAmount();


    function sell(address token, address affiliate, uint256 amount, uint256 expectedOut)
        external
        nonReentrant
        returns (uint256)
    {
-       _validateToken(token);
-
        if (factory.PROTOCOL_PAUSED()) revert ProtocolPaused();
        if (amount == 0) revert InvalidAmount();
```